SimpAgent 标书文案与流程图提示词

1. 项目框架梳理

1.1 一句话定位

SimpAgent 是一个面向复杂任务场景的智能体（Agent）运行与调试框架，核心特点是“可观测、可中断、可恢复、可分叉、可热更新”，能够将多 Agent 协作流程从“黑盒生成”升级为“可审计、可干预、可迭代优化”的工程化系统。

1.2 当前系统已具备的核心能力（基于本仓库现状）

- 多 Agent 工作流运行时（基于 LangGraph.js）
- SQLite 持久化与 checkpoint 能力（支持历史回看）
- 运行中断 / 恢复 / 人工介入
- 按 checkpoint 进行状态补丁（state patch）
- 按 checkpoint 分叉重跑（fork），便于调参与方案对比
- PromptBlock 结构化提示词编译（不是简单拼接 system prompt）
- 多模型厂商兼容调用层（OpenAI Chat / OpenAI Responses / Gemini OpenAI-compatible / Mock）
- 工具执行运行时（ `function` / `shell` ，含白名单与超时控制）
- Trace 事件总线（SQLite 存储 + WebSocket 实时订阅 + 断线补发）
- HTTP 控制面 + WS 实时调试面分离设计

1.3 架构分层

1. 控制面（HTTP API）
   提供创建运行、暂停、恢复、中断、历史查询、状态补丁、Prompt 覆盖、分叉运行、配置管理等接口，承担“命令与查询”职责。
2. 运行时内核（LangGraph.js + FrameworkRuntimeEngine）
   负责多节点工作流执行、checkpoint 管理、状态推进、断点安全点中断、恢复继续执行，是系统的核心执行层。
3. 提示词工程层（PromptCompiler）
   将 PromptBlock、上下文、工具描述、任务输入按规则编译成结构化消息，并记录 PromptTrace，使提示词过程可解释、可回溯、可优化。
4. 模型兼容层（UnifiedProviderClient）
   将不同模型厂商接口差异统一为一致的请求/响应语义，降低后续切换模型、扩展模型的集成成本。

5. 工具运行时（ToolRuntime）
   统一抽象工具契约，支持函数工具与受控 shell 工具，强调权限约束、超时控制、执行审计，便于安全落地。
6. 可观测与调试层（TraceEventBus + WS）
   将节点执行、Prompt 编译、模型调用、工具调用、路由决策等事件写入存储并实时推送，实现“运行过程可看见、异常可定位、链路可回放”。

2. 可直接放进计划书的文案

2.1 项目简介（短版，约 120-180 字）

本项目拟构建一套面向复杂任务处理的智能体协作框架，重点解决当前大模型应用“可用但不可控、能生成但难调试、可演示但难工程化”的痛点。系统以多 Agent 工作流为核心，融合可观测追踪、断点中断/恢复、checkpoint 历史回放、状态补丁与分叉重跑等能力，实现从任务编排、提示词装配、模型调用到工具执行的全链路透明化。该框架可作为科研验证平台与工程落地底座，适用于教学实验、智能应用原型开发和后续产业化扩展。

2.2 项目简介（中版，约 250-350 字）

SimpAgent 面向“多智能体协同 + 工程可控”场景，设计并实现了一套可观测、可中断、可恢复、可热更新的智能体运行框架。与常见仅关注模型输出结果的 Demo 不同，本项目强调运行过程的透明化与可干预性：系统通过 LangGraph.js 提供 checkpoint、状态历史与恢复执行能力，在此基础上构建了统一运行时引擎，支持工作流级别的暂停/恢复、人工中断、安全点断点、状态补丁和分叉重跑。与此同时，项目引入结构化 PromptBlock 编译机制，将提示词配置从“单段文本”升级为“可组合、可追踪、可审计”的模块体系；通过统一 Provider 兼容层，实现对 OpenAI Chat/Responses 及 Gemini 兼容接口的统一接入。系统还提供 HTTP 控制面与 WebSocket 实时调试通道，便于构建可视化调试工具。该方案兼具研究价值（便于开展智能体决策与提示词策略实验）与应用价值（为企业级智能体应用提供工程底座）。

2.3 项目优势（计划书“项目特色与优势”段落）

本项目的突出优势在于，它不是单纯追求“生成结果”的智能问答工具，而是面向真实开发与迭代场景构建的智能体工程底座。系统能够记录并展示多 Agent 执行过程中的关键事件，包括节点路由、提示词编译、模型调用、工具执行与异常信息，使开发者能够快速定位问题来源；同时支持在 checkpoint 上进行状态补丁与分叉重跑，大幅降低调试成本和试错成本。项目采用模块化分层设计，运行时内核、提示词编译器、模型兼容层、工具运行时、可观测总线相互解耦，便于后续扩展新模型、新工具和新工作流。相比传统“一次生成即结束”的应用形式，本项目更适合作为科研实验平台、课程实践平台以及智能体产品原型的基础设施。

2.4 创新性（条目版，适合“创新点”栏目）

1. 提出并实现“可观测 + 可中断 + 可分叉”的智能体运行框架形态，将大模型应用从黑盒执行提升为可审计、可干预流程。
2. 将提示词系统设计为结构化 PromptBlock 编译流水线，支持插槽、优先级、触发条件、覆盖补丁与 PromptTrace，增强提示词工程的可解释性与可维护性。
3. 在统一运行时中融合 checkpoint 历史、状态补丁与分叉重跑能力，使智能体调试具备“版本化实验”特征，适合科研迭代与策略比较。
4. 构建统一 Provider 兼容层，屏蔽不同模型 API 的差异，降低模型切换成本，提升系统持续演进能力。
5. 采用 HTTP 控制面与 WebSocket 调试面分离架构，实现“命令控制”和“实时观测”解耦，利于前端调试平台和后续多端接入。

2.5 创新性（段落版，适合“技术创新与方法创新”栏目）

本项目的创新性主要体现在“智能体工程化可控”这一方向。传统大模型应用往往以提示词 + 单轮调用为主，缺乏对执行过程的精细化观察与干预机制，导致系统难以稳定迭代。本项目在运行时层面引入 checkpoint、恢复执行、人工中断与分叉重跑，使智能体运行过程具备可追踪、可回放、可修正能力；在提示词层面采用 PromptBlock 结构化编译机制，并生成 PromptTrace，从而提升提示词策略实验的可解释性；在系统架构层面通过模型兼容层与工具运行时抽象，实现多模型、多工具的统一接入与安全控制。该设计兼顾科研实验需求与工程落地需求，具有较强的推广潜力。

2.6 可行性与实施基础（适合“研究基础/实施条件”栏目）

项目当前已完成后端核心骨架的可运行版本，包含运行时引擎、HTTP API、WebSocket 调试通道、SQLite 配置与 Trace 存储、默认多 Agent 工作流、Prompt 编译器与工具运行时，并已通过本地构建与最小冒烟测试。这意味着项目并非停留在概念设计阶段，而是已经具备可验证、可演示、可继续扩展的工程基础。下一阶段可在现有协议之上快速补齐可视化调试前端、完善更多测试场景（如中断恢复、断线重连、工具权限策略）并推进面向实际业务场景的适配。

2.7 应用价值与推广前景（适合“应用前景/社会经济效益”栏目）

该框架可服务于高校科研训练、课程教学实践、创新创业项目原型开发及企业智能体应用验证等场景。在教学层面，系统能够把原本抽象的多 Agent 运行过程可视化，降低学习门槛；在科研层面，支持通过 checkpoint 分叉与状态补丁进行策略对比实验，提高实验效率；在应用层面，统一模型接入与工具执行机制有利于快速适配不同厂商模型与业务工具，缩短原型到产品的周期。随着智能体应用对“稳定性、可解释性、可审计性”的要求不断提升，本项目具有较好的持续迭代与推广价值。

3. 标书写作时可替换的表达

3.1 面向“大创/创新创业计划”的表述建议

- 关键词偏向：工程落地、原型验证、平台化能力、可扩展、应用场景、商业转化潜力
- 建议强调：可快速接入新模型/新工具、调试效率高、适合做行业原型和课程竞赛项目底座

3.2 面向“科研/自然科学类项目”的表述建议

- 关键词偏向：实验平台、可复现、可控变量、可解释、策略对比、方法验证
- 建议强调：checkpoint 分叉重跑、PromptTrace、状态补丁、人机协同干预、过程审计

3.3 一句“防追问”说明（答辩时很好用）

当前版本已完成后端核心能力闭环，前端调试界面可基于现有 HTTP/WS 协议快速实现，因此项目具备明确的阶段性成果与持续迭代路径。

4. 流程图生图提示词（给生图 AI 用）

4.1 中文主提示词（推荐）

请生成一张“多智能体可观测运行框架”的专业流程图海报风信息图，风格用于高校项目计划书/科研申报书插图。画面要求简洁、科技感、可信赖、白底或浅色底、蓝青色主色、少量橙色高亮、矢量扁平风、线条清晰、模块边框统一、中文标签可读、布局严谨、适合 A4 纸插图。
流程结构从左到右（或上到下）清晰展示：
用户请求 -> HTTP 控制面（创建运行/暂停/恢复/中断） -> 运行时引擎（LangGraph 工作流执行） -> 节点执行（Orchestrator / Worker / Reviewer） -> Prompt 编译器（PromptBlock 装配、触发条件、PromptTrace） -> 模型兼容层（OpenAI Chat / Responses / Gemini Compatible） -> 工具运行时（Function Tool / Shell Tool，白名单与超时控制） -> Trace 事件总线（事件记录、实时推送） -> SQLite 存储（配置版本、checkpoint、trace） -> WebSocket 调试端（实时订阅、断线补发） -> 人工干预（状态补丁、Prompt 覆盖、Fork 分叉重跑） -> 回到运行时引擎继续执行。
请用明显的箭头表示主链路，并用虚线表示“调试/回放/干预”链路；在图中突出 4 个标签徽章：可观测、可中断、可恢复、可分叉。整体视觉要专业、可用于正式标书，不要卡通化，不要杂乱背景，不要透视变形，不要难以辨认的小字。

4.2 英文增强版提示词（部分模型更稳定）

Create a clean professional system architecture flowchart infographic for a university innovation project proposal. Theme: "Observable Multi-Agent Runtime Framework". Use a white or very light background, blue-cyan primary palette with subtle orange highlights, flat vector style, clear connectors, consistent rounded rectangles, readable Chinese labels, high information density but uncluttered layout, publication-ready for A4 report insert.
Main flow: User Request -> HTTP Control Plane (create run / pause / resume / interrupt) -> Runtime Engine (LangGraph workflow execution) -> Agent Nodes (Orchestrator / Worker / Reviewer) -> Prompt Compiler (PromptBlock assembly, triggers, PromptTrace) -> Model Provider Compatibility Layer (OpenAI Chat, OpenAI Responses, Gemini-compatible) -> Tool Runtime (Function Tool, Shell Tool with whitelist + timeout) -> Trace Event Bus (record + realtime dispatch) -> SQLite Storage (versioned configs, checkpoints, traces) -> WebSocket Debug Console (subscribe, heartbeat, replay) -> Human-in-the-loop Intervention (state patch, prompt override, fork replay) -> back to Runtime Engine.
Use solid arrows for execution flow, dashed arrows for debugging and intervention loops. Add four visual badges: Observable, Interruptible, Recoverable, Forkable. Make it formal, academic, trustworthy, and suitable for grant proposal documents.

4.3 负面提示词（可选，提升出图质量）

避免：3D 透视、赛博朋克霓虹过强、人物插画、卡通风、背景噪点、文字模糊、过多装饰图标、流程方向混乱、箭头交叉严重、字体过小、深色背景大面积压字、英文错拼、手绘风。

4.4 出图参数建议（按常见生图工具调整）

- 画幅比例：`16:9`（做 PPT/汇报）或 `4:3`（做标书插图更稳）
- 分辨率：长边建议 `1600px` 以上（后续可二次放大）
- 风格强度：中等（避免过度艺术化影响可读性）
- 文字清晰度：若模型文字能力弱，可先生成“无字结构图”，再用 PPT/Figma 覆盖文字